

The Viktor Router Lab, explained in plain English

Every panel, tile, chart and slider on the dashboard — what it is, why it exists, and what happens when you change it. No formulas you don't need. Where a number appears, it is the number the committed artifacts produce at default settings.

File: `dashboard/index.html` (the Lab) · `dashboard/present.html` (the deck)

Dataset: 1,025 real trajectories · source `export_linked`

Defaults: method **Blend + cuts**, $\alpha = 0.85$, cuts 0.55 / 0.85, cost model **cache-aware**

1	The 60-second mental model	p. 2
2	The header and the seven score tiles	p. 3
3	Left panel — Router (Part 1)	p. 5
4	Left panel — Evaluator (Part 2)	p. 8
5	Left panel — Cost model & pricing	p. 10
6	The eight live charts	p. 11
7	Pipeline findings (the three fixed cards)	p. 14
8	Honesty notes — what these numbers cannot tell you	p. 16
9	Experiments to try, and what you should see	p. 17

1 The 60-second mental model

The whole dashboard exists to answer one question: **can you tell how hard a task is before you start it, well enough to safely send the easy ones to a cheap model?**

Three moving parts

The router — "Part 1"

- WHAT** Looks at a task *before* any work happens: the first user message, the system prompt, and the list of tools available. Nothing else. It outputs a tier — **T1** cheapest, **T2** middle, **T3** strongest.
- WHY** This is the actual product. In production you must choose a model before you know how the task will go.

The evaluator — "Part 2"

- WHAT** Reads the *whole finished* trajectory and grades how hard the task turned out to be — D1, D2 or D3 — from effort signals: how many tools were called, how many tokens were generated, how many errors, how long the retry loops ran.
- WHY** It is the answer key. The router never sees it, so comparing them is a real test rather than a self-assessment.

The comparison — everything on the right

- WHAT** Router tier vs observed difficulty, task by task, plus what each choice would have cost.
- WHY** A router is only interesting if it keeps quality *and* saves money. One without the other is trivial.

The one word you need: "served"

A task is **served** when `router tier ≥ observed difficulty` — the model it got was at least as strong as the job needed. Over-shooting (T3 on an easy task) still counts as served; it just wastes money. Under-shooting is the failure mode that hurts users.

Why every knob is live

Nothing on this page is a screenshot. Move any slider and all seven tiles, all eight charts and the frontier recompute from the raw 1,025 trajectories in about 70 ms. That is deliberate: it lets a sceptic try to break the result instead of taking our word for it.

The one thing that is *not* live

The machine-learning probabilities (used by three of the five methods) were trained once, in the Python pipeline, against the evaluator's *default* settings. They are out-of-fold — grouped by workspace, so no task's prediction ever saw its own outcome — but they are frozen. If you change the Evaluator panel, the ML numbers are now slightly stale; rerun `run_pipeline.py` to retrain. The heuristic method has no such caveat.

2 The header and the seven score tiles

Header chips (the grey pills under the title)

Five standing facts about the run, kept visible so no chart can be quoted out of context: the trajectory count, the data source, that tokens are `chars / 4` estimates, that the router never sees which model actually ran, and that ML probabilities are out-of-fold.

The tiles

Seven headline numbers. Values below are at default settings. The first three measure *skill*, the next three measure *safety*, the last measures *money*.

Exact agreement — 65.3%

- WHAT** Share of tasks where the router's tier is exactly the observed difficulty.
- WHY** The most intuitive score. It is also the most misleading one on its own.
- CAREFUL** Do **not** compare it to 33%. About 55% of tasks are D1, so a router that blindly said "Tier 1, always" would score ~55% and be useless. 55% is the floor to beat, not chance.
- MOVES WHEN** Any router knob changes; also when you move the evaluator cuts, because that redefines the answer key.

Adjacent agreement — 97.1%

- WHAT** Share within one tier of correct.
- WHY** Both sides are chopping a smooth continuum into three buckets, so a task sitting right on a boundary can fall either side for no meaningful reason. 97% means the router essentially never makes a *wild* mistake — the errors are all boundary noise.
- MOVES WHEN** Barely ever. If it drops below ~90% something is genuinely broken.

Rank correlation — $\rho = 0.693$ (Spearman)

- WHAT** Correlation between the router's raw score and the evaluator's raw score, ignoring tiers entirely.
- WHY** It is threshold-free. Tiles 1 and 2 depend on where you drew the cuts; this one measures whether the router *ranks* tasks correctly regardless of cuts. If the cuts are wrong you can fix that; if ρ is low there is no signal to work with.
- MOVES WHEN** Feature weights or α change. Cut sliders do **not** move it — a useful sanity check that you are looking at the right thing.

Under-routed — 17.8%

- WHAT** Tasks sent to a tier *below* what they needed.
- WHY** This is the only number that represents real user harm. Everything else is accounting.
- MOVES WHEN** Spending more pushes it down. The **τ -sufficiency** method exists specifically to control it: raise τ and this falls.

Served — 82.2%

- WHAT** The complement of under-routed: tasks that got at least the capability they needed. Counted per task, every task worth 1.
- WHY** The headline quality metric — "how often did we not let the user down".

Weighted served — 68.8%

WHAT	The same thing, but each task counts in proportion to its size (its Tier-3 cost) instead of counting as 1.
WHY	This is the honesty tile. A router can look excellent per task by nailing hundreds of tiny questions while quietly failing the handful of enormous jobs — the ones that carry most of the tokens and most of the user's frustration. Weighting by size makes those failures visible.
READ THE GAP	82.2% vs 68.8% is a 13-point gap. It says the tasks we under-serve are systematically the <i>big</i> ones. That is expected — big tasks are the hard ones — and it is the number to improve.
WATCH IT WHEN	You switch to λ-Bayes . That method optimises tasks-per-dollar and is known to sacrifice token-heavy work; this tile is where you catch it.

Est. savings — 45.5% (\$147.61 vs \$270.67)

WHAT	What this routing policy would spend versus sending every task to Tier 3, under the currently selected cost model and prices.
WHY	The other half of the deal. Quality with no saving is just "use the big model".
CAREFUL	Prices are assumptions (the model IDs in the dataset are anonymised) and tokens are <code>chars / 4</code> estimates. Treat the <i>ratio</i> as meaningful and the absolute dollars as illustrative.

3 Left panel — Router (Part 1)

This panel decides how a task gets turned into a tier. It has two layers: pick a **method**, then tune that method's knobs. Sliders that do not apply to the active method hide themselves, so the panel never shows you a control that does nothing.

3.1 The five methods

METHOD	HOW IT DECIDES	WHY IT IS ON THE PAGE
Blend + cuts the default	Mix the ML difficulty ranking with the hand-built heuristic ranking ($\alpha = 0.85$ ML), then cut the result at the 55th and 85th percentiles.	It won the cross-validation sweep — best balance of agreement and <i>token-weighted</i> quality. This is what we would actually ship.
Heuristic cuts	The hand-built score alone: four signed feature groups, percentile cuts. No learning, no labels.	The interpretable baseline. If the blend barely beats this, the ML is not earning its complexity. It also still works on day one with zero training data.
ML τ-sufficiency	For each task, take the cheapest tier whose probability of being sufficient is at least τ .	The cheapest way to hit a quality <i>target</i> . Use this when someone says "I need 90% served — what does that cost?"
ML λ-Bayes	Pay for a stronger tier only when $\lambda \times P(\text{too hard})$ exceeds the price gap.	Maximum tasks-per-dollar. Included partly as a cautionary tale — watch Weighted served collapse while Served stays high.
K-Means (k=3)	Unsupervised clustering of the routing features into three groups, ordered by mean score.	The null hypothesis: maybe you don't need difficulty at all, just structure? You do. It is measurably dominated, and showing that is the point.

3.2 The dispatch knobs

α — ML share of the blend — default 0.85, visible for Blend only

WHAT	Blend = $\alpha \times \text{ML ranking} + (1-\alpha) \times \text{heuristic ranking}$.
WHY 0.85	It won the sweep: near-maximum agreement while the heuristic still holds enough weight to protect big tasks.
TURN IT TO 0	You get pure heuristic — identical to the "Heuristic cuts" method.
TURN IT TO 1	Pure ML. Exact agreement ticks up slightly; weighted served gets worse. That trade is exactly why the default is 0.85 and not 1.0.

τ — confidence to stay cheap — default 0.80, visible for τ -sufficiency only

WHAT	"Only keep a task on a cheap tier if I am at least τ confident that tier is enough."
RAISE IT	Safer and pricier. Under-routed falls, savings fall. At $\tau \rightarrow 1$ you converge on all-Tier-3.
LOWER IT	Cheaper and riskier, converging on all-Tier-1.
USE IT TO	Answer a quality-target question directly, e.g. the cheapest configuration that still serves 90%.

λ — miss penalty — default \$0.30, log-scaled slider, λ -Bayes only

WHAT	How many dollars a quality miss is worth. The rule then spends up only when the expected miss cost beats the price difference.
RAISE IT	Misses hurt more, so it buys more Tier 3.
LOWER IT	It gets stingy fast.
KNOWN FLAW	It counts every task as one task, so it happily starves a few huge jobs to rescue many small ones. Keep an eye on the Weighted-served tile the whole time you play with this.

3.3 The four feature-group weights

These build the heuristic score. Two *add* difficulty, two *subtract* it — the sign is printed on the slider label, and it is a finding, not a knob choice.

GROUP	SIGN	DEFAULT	WHAT IT MEASURES, AND WHY THE SIGN
ask	+	1.50	Density of the request itself — named entities, questions, action verbs, "and then" coordination, URLs, attachments, code blocks, length. A dense ask is a hard task. Heaviest weight because it is the strongest honest signal.
harness	+	0.70	How heavily configured the workspace is — system-prompt sections, flags, skills, whether a subagent tool exists. Elaborate setups accompany serious work.
breadth	−	0.80	Size of the generic toolset. Counter-intuitive but consistent in the data: a huge everything-enabled toolbox goes with quick conversational turns, so it marks tasks <i>easier</i> .
midthread	−	0.80	Marks that we joined a conversation late — truncated context, auto-read blocks. Most of the work already happened, so what remains is usually small.

What happens if you zero one

Set a weight to 0 and that whole family of evidence is switched off. Zeroing **ask** is the most destructive — rank correlation drops sharply. Zeroing **breadth** or **midthread** is instructive: it shows how much of the router's skill comes from knowing what marks a task *easy*, not just hard.

3.4 The two tier cuts

Tier 1 | 2 cut and Tier 2 | 3 cut — defaults 0.55 and 0.85 (percentiles)

WHAT	Where the score line is chopped. Below the first cut → Tier 1; above the second → Tier 3.
WHY PERCENTILES	They pin the tier <i>mix</i> — 55% / 30% / 15% — so cost is predictable and the router's mix matches the evaluator's by construction. Comparisons then test ordering skill rather than accidental budget differences.
MOVE THEM RIGHT	Cheaper, more under-routing.
MOVE THEM LEFT	Safer, more spend. Sliding the Tier-3 cut is literally how the frontier curve for this method is traced.
NOTE	The two sliders cannot cross; the panel clamps them and shows the value that was actually applied.

3.5 Two utilities

k-means seed — appears only for K-Means

WHAT	Random start for the clustering.
WHY EXPOSED	Honesty. Change it and the clusters shift a little. A method whose answer depends on a random seed should have to admit it.

Export tiers CSV

WHAT Downloads a per-task tier assignment.

WHY IT MATTERS The file's header comments record every knob setting that produced it — method, α , τ , λ , weights, cuts, overrides, prices. A bare number dump would not be reproducible; this is.

4 Left panel — Evaluator (Part 2)

This panel builds the answer key. It reads finished trajectories and scores effort. Editing it changes what "hard" means — so it changes every score on the page, including scores you might have thought were fixed.

Read this before touching anything here

The evaluator is deliberately rule-based, not learned. A learned grader trained on the same trajectories the router sees would leak information and quietly inflate every result. Rules are auditable: you can read them, disagree with them, and change them right here — which is the point of putting them on screen.

4.1 The ten effort weights

Each metric is converted to a percentile rank, then combined with these weights. Ranks rather than raw values, so one 400-tool-call outlier cannot dominate the scale.

METRIC	WEIGHT	WHAT IT IS, AND WHY IT CARRIES THAT MUCH
<code>n_tool_calls</code>	0.22	How many tools the agent invoked. The single strongest effort signal, hence the top weight.
<code>n_llm_calls</code>	0.18	How many model turns the task took. Rounds of thinking.
<code>gen_tokens</code>	0.16	Tokens the model itself produced — text plus tool arguments. Volume of output work.
<code>context_tokens</code>	0.10	How much the final call had to read. Accumulated state.
<code>n_tool_errors</code>	0.10	Results containing errors or tracebacks. Measures <i>struggle</i> , not just length — a short task full of errors was genuinely hard.
<code>n_distinct_tools</code>	0.08	How many different tools. Breadth rather than repetition.
<code>tool_output_tokens</code>	0.06	Volume the agent had to digest.
<code>max_repeat_streak</code>	0.04	Longest run of the same tool back to back — a retry/stuck-loop signal.
<code>n_user_turns</code>	0.03	Follow-ups and nudges. Weak: it conflates a chatty user with a hard task.
<code>reasoning_tokens</code>	0.03	Visible reasoning summaries. Deliberately tiny — only the GPT family logs these , so a large weight would grade providers instead of tasks.

The trap in this table

Raise `reasoning_tokens` and you are no longer measuring difficulty; you are measuring which vendor served the task. That is a great way to manufacture an impressive-looking result and a completely fake one. The weight is small on purpose and the tooltip says so.

4.2 The two difficulty cuts

D1 | D2 cut and D2 | D3 cut — defaults 0.55 and 0.85

WHAT	Percentiles splitting the effort score into three difficulty classes. This is the source of the "55% of tasks are D1" fact quoted elsewhere.
WHY THEY MATCH THE ROUTER CUTS MOVE THEM	Same values on both sides means neither side gets a free win from a friendlier mix. Both sides' scores move. Widening D1 makes the task set look easier and inflates served %; narrowing it makes everything look harder. If you want to stress-test the headline claim, this is the strongest lever on the page — try it, and note that the router's <i>ranking</i> skill (p) survives.

4.3 The two hard overrides

tool errors $\geq 5 \rightarrow$ force D3 · tool calls $\geq 40 \rightarrow$ force D3

WHAT	Two absolute rules that bypass the score entirely.
WHY	Percentiles are relative. In a dataset of mostly-easy tasks, a genuinely brutal task can still land mid-pack. These two rules say: 40 tool calls or 5 errors is objectively hard, no matter what everything else looked like.
RAISE THE THRESHOLDS SET THEM VERY LOW	Fewer forced D3s, the grading gets softer, served % rises slightly. Almost everything becomes D3 and the whole exercise collapses — which is a useful thing to see once.

5 Left panel — Cost model & pricing

Quality is half the story. This panel decides what money means, and it contains the single most important methodological point in the project.

5.1 Cache-aware vs cache-blind

Cache-aware — the default

WHAT Charges the shared conversation prefix at the discounted cached rate on every call after the first (roughly $T \times (n-1) / 2$ tokens replayed).

WHY DEFAULT It is how providers actually bill. A long task re-reads its own history dozens of times, and pricing that at full rate is simply wrong.

Cache-blind

WHAT Context $\times$ input price + generated $\times$ output price. No caching at all.

WHY IT IS HERE Not as an alternative — as an exhibit. Under cache-blind accounting, a policy that switches models mid-task looks brilliant, because switching appears free. It isn't: switching models throws the cache away and you pay for the whole history again.

SWITCH TO IT AND Estimated savings jump (\$147.61 $\rightarrow$ \$80.37 of spend). That jump is the illusion. The "Cache trap" card on page 15 puts a number on it.

5.2 The nine price fields

Three numbers per tier — input, cached, output — in dollars per million tokens.

TIER	INPUT	CACHED	OUTPUT	ROLE
T1	0.20	0.02	1.20	Small/fast class
T2	2.00	0.20	10.00	Mid class
T3	5.00	0.50	25.00	Frontier class

Why these are editable at all

BECAUSE The dataset's model IDs are anonymised, so exact prices are unknowable. Rather than hide that behind a constant, the assumption is exposed as a control.

CHANGE THEM AND Every cost number and the whole frontier redraw. The finding that matters is that the *shape* is robust: the committed artifacts also report a Tier-3 price sensitivity check — repricing Tier 3 upward moves savings from 45.5% to 53.6%, i.e. the conclusion strengthens rather than breaks.

COMPRESS THE GAPS Make all three tiers cost the same and routing becomes pointless — savings go to zero. That is the correct behaviour, and worth seeing once to confirm the cost model is actually wired to the charts.

6 The eight live charts

1 • Router score — histogram

WHAT	Distribution of the routing-time difficulty score, coloured by the tier each region maps to. Dashed lines are the cuts.
WHY	Shows the shape of the decision. A healthy score spreads out; a score piled up in one spike means the cuts are separating noise.
CHANGES WHEN	Weights reshape the distribution; cuts move the dashed lines and recolour the bars. For the τ and λ methods the subtitle changes too, because tiers then come from the dispatch rule rather than from cuts.

2 • Evaluator difficulty score — histogram

WHAT	The same view for observed difficulty.
WHY	Put the two histograms side by side and you can see whether the router is even trying to reproduce the right shape.

3 • Tier mix — router vs evaluator — paired bars

WHAT	How many tasks each side puts in each tier.
WHY	The fairness check. For cut-based methods the bars match by construction — good, that is the control. For τ and λ they float free, and a big mismatch tells you the method is buying quality with budget rather than with skill.

4 • Confusion matrix — 3×3 heatmap

WHAT	Rows are the router's decision, columns are what the trajectory revealed. Diagonal = agreement.
WHY	It separates the two kinds of error, which the tiles cannot. Below the diagonal is under-routing (risk); above is over-routing (waste).
AT DEFAULTS	452 / 141 / 76 on the diagonal. The two worst cells are 98 (T1 chosen, D2 needed) and 96 (T2 chosen, D1 was enough) — boundary confusion between adjacent classes, which is the good failure mode. Only 14 tasks are the severe T1-on-D3 error.

5 • Cost-quality frontier — the headline chart

WHAT	Money on the x-axis (as a share of the all-Tier-3 bill), quality kept on the y-axis. The curve is traced by sweeping the active method's <i>own</i> knob — τ , λ , or the Tier-3 cut.
WHY	Because a single point proves nothing. Anyone can hit a good served % by spending more. A curve shows the whole trade-off, and you can only claim a router is good if its curve sits above the alternatives everywhere.

Five things are drawn on it. Reading them correctly is most of the argument:

MARK	WHAT IT IS	WHY IT IS THERE
■ Blue line	Our router, served per task.	The optimistic read.
■ Orange line	Our router, token-weighted served.	The pessimistic read of the <i>same</i> policy at the <i>same</i> cost. A real result has to beat the baselines on both lines, not just the flattering one.
Two dashed greys	Random routing, expected — one dash pattern per quality metric.	The honest floor. Because 55% of tasks are D1 and <i>any</i> tier serves them, random already looks decent — so the real separation lives in the $\geq 90\%$ -served region and in the weighted metric. Saying this out loud is more useful than a flattering baseline.
● Green dot	The logged dispatch — what actually ran in the dataset, repriced through our inferred tier map.	The real-world comparison: not "vs a strawman", but "vs what the production system did".
Hollow dot	Always top tier: 100% served at 100% cost.	The trivial policy everyone starts from. Drawn hollow so it cannot be mistaken for one of our series.

The fairness fix hidden in the green dot

Tool errors and retry streaks were used to *infer* which models are strong. If we then also graded difficulty using those same two metrics, a model that logged errors would be assigned a low tier **and** have its tasks graded harder — mechanically depressing the logged system's score and flattering ours. So for this one comparison only, difficulty is regraded with those two weights zeroed and the error override off, and our policy is re-scored the same way. The tooltip reports both under that identical grading, plus a sensitivity check that demotes the least-stable Tier-3 model. It would have been easy not to do this. The chart would have looked better.

6 • Router score vs observed difficulty — scatter, 1,025 dots

WHAT	One dot per task: predicted on x, observed on y, coloured matched / under-routed / over-routed .
WHY	Aggregates hide failure modes; a scatter cannot. Hovering any dot gives its trigger, a task preview, both scores, the sufficiency probabilities, its tool-call and error counts, and which model actually ran it.
WHAT TO LOOK FOR	The orange dots in the bottom-right region — tasks that looked trivial going in and turned out to be enormous. Those are the genuinely hard cases, and no amount of tuning fixes them, because the information was not available at routing time.

7 • Routing-time signal strength — tornado bars

WHAT	Every individual routing-time feature, ranked by its correlation with observed difficulty. Blue to the right = marks tasks hard; red to the left = marks them easy.
WHY	This is the interpretability layer — the answer to "is there actually signal here, and can you name it?" It also justifies the negative signs in the feature-group table: you can see the left-pointing bars directly.
NOTE	These correlations are properties of the data, so they move with the evaluator weights but not with any router knob.

8 · Largest disagreements — table, top 15

WHAT	The 15 tasks where the router and the trajectory disagree most, sorted by tier gap and then by score gap, with previews and the model that ran each one.
WHY	Aggregate metrics tell you <i>how much</i> you are wrong; this tells you <i>where</i> . It is the debugging surface — and publishing your worst 15 cases rather than your best is the point.
CHANGES	Any knob moves, because the ranking is recomputed. If the same task IDs keep appearing
WHEN	across very different settings, that task is genuinely unpredictable rather than mis-tuned.

7 Pipeline findings — the three fixed cards

These three cards do **not** react to the sliders. They come from committed artifacts produced by the Python pipeline, because each one needs bootstrapping or a held-out split that cannot honestly be recomputed in a browser on every keystroke. To refresh them you rerun the scripts.

7.1 Inferred model tiers

- WHAT** The dataset's model IDs are anonymised, and no capability ranking is given. So we infer one: inside *matched difficulty buckets*, compare error rates and retry streaks (a "struggle" z-score) and rank the models. Then bootstrap 200 resamples to see how stable each placement is.
- WHY** Without this there is no way to reprice the logged dispatch — you could not draw the green dot on the frontier, and the whole comparison would be against nothing.
- WHAT IT SAYS** Nine models, terciles. Models with fewer than 15 tasks are flagged low-confidence and inherit a neighbour's tier rather than pretending to a rank. Stability ranges from 99% (very solid) down to 57.5% (a genuine coin-flip at the boundary), and the card prints that number for every row instead of hiding it.

The null result we chose to advertise

Revealed preference across models is **flat** — the difficulty of tasks each model was handed is roughly the same everywhere. In other words the production dispatch was **not difficulty-aware**. That is a negative finding about the data, and it is also the entire business case: that unexploited headroom is what a router captures. Leading with it is stronger than burying it.

7.2 The cache trap

- WHAT** Providers cache the shared prefix across a task's calls. Switch models mid-task and that cache is gone — you re-pay for the whole history. Naive accounting misses this entirely, so per-call switching policies look far better than they are.
- WHY IT IS A HEADLINE FINDING** Because "just pick the cheapest model for each individual call" is the obvious idea, and it is wrong. Anyone who costs it naively will ship it and lose money.

POLICY	NAIVE SAVED	CACHE-AWARE SAVED	SWITCHES
All Tier 3	0%	0%	0
Per task (our router)	96.0%	96.0%	0
Per-call greedy (by size)	48.5%	13.2%	36
Per-call cheap-tool-loops	17.3%	15.1%	25

Read the third row: a claimed 48.5% saving becomes 13.2% once resets are priced. Our per-task policy is the only one whose two columns are identical — it never switches, so it pays no reset *by construction*.

Measured vs modelled — the card labels both

The table above is **measured**, but only on the 25 trajectories with two or more logged calls, and the export samples roughly one call per task — so it hides most switch points and the measured 2.2-point overstatement is a **floor, not the effect**. The card therefore also reports a **modelled** extrapolation over full trajectories, with its three assumptions printed on it: cache resets cost about **\$160**, which is **72%** of the all-Tier-3 input budget — turning a claimed 17.3% saving into **−55%**, i.e. more expensive than simply using the best model for everything. Keeping the measured and modelled numbers visibly separate is the honest way to present this.

7.3 Matched cross-model check

WHAT	The natural experiment hiding in the data. Tasks of similar routing-time difficulty happened to run on different models. So: do weak models struggle <i>disproportionately</i> on hard tasks? That is a difference-in-differences question, and it is testable.
WHY IT MATTERS MORE THAN IT LOOKS	The entire "served" metric rests on an assumption: that under-routing actually hurts. If a weak model handles hard tasks just as well, served % is meaningless and the project has no point. This card is where that assumption is put at risk.
THE DESIGN	The tier map is fitted on the even-numbered half of trajectories and validated on the odd half only — so the check cannot confirm itself. Both interactions are bootstrapped with 5,000 resamples.

	LOW TIER (T1-T2)	HIGH TIER (T3)
Easy tasks	6.0% errors · 3.35 streak (n=81)	5.1% errors · 1.95 streak (n=226)
Hard tasks	12.3% errors · 6.70 streak (n=82)	4.7% errors · 2.57 streak (n=123)

Verdict: SUPPORTED on both metrics

The extra penalty a weak model pays specifically for being handed a hard task is **+6.7 percentage points** of error rate (95% CI 2.5 to 10.8) and **+2.73** on retry streak (95% CI 1.56 to 3.99). Both intervals exclude zero, so both are significant. Under-routing has a measurable cost — which means "served" is measuring something real, not an accounting convention.

8 Honesty notes — what these numbers cannot tell you

This is the footer of the dashboard, reproduced because it is the most important text on the page. Every one of these is a real limitation, stated by us rather than found by a reviewer.

Difficulty is an *effort* proxy, not true difficulty

MEANING A task can be long and easy, or short and genuinely hard. We measure how much work happened, which is correlated with difficulty but is not the same thing.

The logged model shapes the effort we measure

MEANING Circularity: a weaker model retries more, which inflates the effort score, which makes the task look harder. Section 7.3's split-half design is what keeps this from contaminating the main claim, and the frontier's green dot is graded with the two affected metrics removed. It is mitigated, not eliminated.

Token counts are `chars / 4` estimates

MEANING The export contains no usage data. Every dollar figure inherits this approximation.

Prices are assumed

MEANING Model IDs are anonymised. Cost *ratios* are the trustworthy part; absolute totals are illustrative. The price fields are editable so you can check that yourself, and the committed sensitivity check shows the conclusion holding under a different Tier-3 price.

The cache-aware model assumes one tier per task

MEANING Under this model no policy pays a cache-reset penalty, which flatters the switching policies. The modelled extrapolation in section 7.2 exists precisely to price what this model leaves out.

λ -Bayes sacrifices token-heavy tasks

MEANING A known defect of that method, not a bug in the dashboard. The weighted-served tile and the orange frontier line are the instruments that make it visible.

ML probabilities are frozen against the evaluator's defaults

MEANING They are out-of-fold and honest, but they were trained once. Change the Evaluator panel and rerun `run_pipeline.py` before quoting the ML methods.

`reasoning_tokens` exists only for one model family

MEANING Give it real weight and you are grading vendors, not tasks.

9 Experiments to try, and what you should see

The fastest way to trust the dashboard is to try to break it. These are the experiments worth running, with the outcome you should expect. If any of them behaves differently, something is wrong — and that is worth knowing.

DO THIS	EXPECT	BECAUSE
Switch to K-Means	Agreement and served both drop clearly.	Clustering finds workspace structure, not difficulty. The null hypothesis, visibly dominated.
Switch to λ-Bayes and watch the tiles	Served holds up; Weighted served falls .	The clearest demonstration on the page of why one metric is never enough.
Set $\alpha = 0$	You land exactly on the Heuristic-cuts result.	Confirms the blend is a genuine mixture and shows how much the ML actually contributes (less than you would guess).
Push τ toward 0.99	Under-routing approaches zero; savings evaporate.	Traces the safe end of the frontier. Useful for answering "what does 95% served cost?"
Flip to cache-blind	Estimated spend drops from \$147.61 to \$80.37.	The size of the illusion the cache trap describes.
Zero the ask weight	Rank correlation drops sharply.	Identifies where the router's skill actually lives.
Zero breadth and midthread	A smaller but real degradation.	Shows that knowing what marks a task <i>easy</i> matters as much as spotting hard ones.
Raise reasoning_tokens to maximum	Numbers shift, and the result becomes meaningless.	The one knob that lets you fake a good result. Worth doing once to understand why its default is 0.03.
Slide the D1 D2 cut right	Served % inflates; p barely moves.	Separates real ranking skill from a friendlier answer key — the strongest stress test available.
Make all three tiers the same price	Savings go to zero.	Confirms the cost model is genuinely wired to the charts rather than baked in.

The companion file: `dashboard/present.html`

Six keyboard-navigated slides that tell the same story in argument order, with every number pulled live from the same artifacts — no figure is retyped, so the deck cannot drift from the lab. The frontier slide animates the knob replay, which is the single most convincing thing to show someone in ten seconds.

If you remember five things

1. The router only ever sees pre-task information — that constraint is the whole point. **2.** "Served" means the model was strong enough; the token-weighted version is the honest one. **3.** The frontier, not any single number, is the result. **4.** Per-call model switching is a trap: cache resets can make it cost more than never routing at all. **5.** Every knob is live so you can attack the result yourself — and the three fixed cards are fixed precisely because faking them live would be dishonest.